Cuestionario Técnico de Control: Recursividad, Arreglos y Punteros en Memoria RAM

Sofia Mojica, Gabriel Zacarías, Mateo Roberts y Leonel De Armas

Escuela de Educación Secundaria Técnica N.° 1 (EEST N.° 1)

LPR 2026: Lenguaje de Programación

Actividad N.° 6: Proyecto Integrador de Área (P.I.A.) [P3_HQ_2.0]

Profesores: Elías York y Yamil Ganduglia

2026

Cuestionario Técnico de Control: Recursividad, Arreglos y Punteros en Memoria RAM

El presente informe tiene como objetivo responder el Cuestionario Técnico de Control correspondiente a la Actividad N.° 6, elaborado en forma grupal por cuatro integrantes. El trabajo aborda cuatro problemáticas centrales del funcionamiento de un programa a bajo nivel: el comportamiento de la memoria RAM ante una recursividad sin caso base, la optimización que aporta la instrucción `break` durante una búsqueda en arreglos, la comprobación de la contigüidad de celdas de memoria mediante el operador de dirección (`&`) y el rol de las variables temporales en el intercambio de valores mediante punteros. Finalmente, se incluye una autoevaluación grupal sobre el reto que resultó más complejo de comprender a bajo nivel.

Distribución de tareas

A continuación se detalla la distribución de tareas específicas de cada integrante dentro del informe (ver Tabla 1).

Tabla 1

Distribución de tareas por integrante

Integrante	Tareas asignadas
Sofia Mojica	Investigación y redacción de la Pregunta 1 (caso base en recursividad); revisión de ortografía, coherencia y normas APA 7.ª edición de todo el documento.
Gabriel Zacarías	Investigación y redacción de la Pregunta 2 (eficiencia de la instrucción `break`); verificación práctica del comportamiento del bucle mediante pruebas de código en el compilador.
Mateo Roberts	Investigación y redacción de la Pregunta 3 (direccionamiento y operador `&`); recopilación y organización de las referencias bibliográficas en formato APA.
Leonel De Armas	Investigación y redacción de la Pregunta 4 (variable temporal en el intercambio de punteros); armado final del documento, portada y exportación del archivo PDF.

Pregunta 1. Caso base: error crítico en la memoria RAM sin caso de corte

Una función recursiva que carece de un caso base de corte nunca detiene su cadena de autollamadas. Cada vez que la función se invoca a sí misma, se reserva un nuevo marco de pila (*stack frame*) dentro del segmento de pila (*stack*) de la memoria RAM, donde se almacenan las variables locales, los parámetros y la dirección de retorno de esa llamada en particular. Como la función jamás alcanza una condición que le indique retornar, estos marcos se siguen apilando uno sobre otro de manera indefinida.

Dado que el segmento de pila tiene un tamaño finito y mucho menor que el de la memoria total del sistema, este proceso agota rápidamente el espacio disponible. El error crítico que se produce se conoce como desbordamiento de pila o *stack overflow*: el programa intenta escribir un nuevo marco en una dirección de memoria que ya está fuera del área reservada para la pila, invadiendo zonas de memoria que no le corresponden. El sistema operativo detecta esta violación de acceso y finaliza el programa de forma abrupta, generalmente mostrando una excepción o un fallo de segmentación (*segmentation fault*), ya que no queda memoria disponible para continuar apilando nuevas llamadas ni para que estas retornen ordenadamente hacia la función que las originó.

Pregunta 2. Eficiencia: optimización del rendimiento mediante la instrucción `break`

Cuando se realiza una búsqueda secuencial dentro de un arreglo, el bucle recorre cada elemento comparándolo con el valor buscado. Si no se utiliza la instrucción `break`, el bucle continúa ejecutando iteraciones y comparaciones hasta llegar al último índice del arreglo, incluso después de haber encontrado el elemento deseado. Esto implica que el microprocesador sigue realizando ciclos de reloj, incrementos del contador, comparaciones y accesos a memoria de forma totalmente innecesaria.

La instrucción `break` interrumpe la ejecución del bucle en el instante exacto en que se cumple la condición de búsqueda, evitando que el procesador siga procesando los elementos restantes del arreglo. En términos de complejidad algorítmica, esto reduce el número de operaciones realizadas en el caso promedio y en el mejor caso, ya que el tiempo de ejecución pasa a depender de la posición en la que se encuentra el dato y no siempre del tamaño total del arreglo. Menos iteraciones significan menos ciclos de reloj consumidos, menos accesos a la memoria caché y, en consecuencia, un menor tiempo de respuesta y un menor consumo energético del microprocesador durante la ejecución de la búsqueda.

Pregunta 3. Direccionamiento: comprobación de contigüidad mediante el operador `&`

El operador de dirección (`&`) permite obtener la dirección de memoria en la que se encuentra almacenada una variable o un elemento de un arreglo. Para comprobar que los elementos de un arreglo ocupan celdas contiguas en la memoria RAM, basta con imprimir la dirección de cada elemento aplicando el operador `&` sobre cada posición, por ejemplo `&arreglo[0]`, `&arreglo[1]`, `&arreglo[2]` y así sucesivamente, generalmente mostrando el resultado en formato hexadecimal.

Al comparar los valores obtenidos, se observa que la diferencia entre la dirección de un elemento y la del siguiente es siempre igual al tamaño en bytes que ocupa el tipo de dato del arreglo (por ejemplo, 4 bytes para un arreglo de tipo `int` o 1 byte para uno de tipo `char`). Esta secuencia de direcciones crecientes, separadas por una diferencia constante, demuestra que el compilador reservó un bloque único e ininterrumpido de memoria para todo el arreglo y que cada elemento se ubica inmediatamente después del anterior. Es precisamente esta contigüidad la que permite calcular la dirección de cualquier elemento a partir de la dirección base del arreglo más el índice multiplicado por el tamaño del tipo de dato.

Pregunta 4. Variables temporales: función en el intercambio de valores con punteros

En un algoritmo de intercambio (*swap*) que utiliza punteros, la función recibe las direcciones de memoria de las dos variables a intercambiar en lugar de sus valores. El problema es que, si se intenta asignar directamente el contenido de una variable a la otra (por ejemplo, `*a = *b`), el valor original de la primera variable se pierde antes de poder ser copiado a la segunda, ya que ambas asignaciones no pueden ocurrir de manera simultánea.

Allí es donde interviene la variable temporal: su función es actuar como un espacio de almacenamiento intermedio que guarda una copia del valor que está a punto de ser sobrescrito. El procedimiento típico es `temp = *a;` luego `*a = *b;` y finalmente `*b = temp;`. De esta manera, la variable temporal conserva el valor original de la primera celda de memoria mientras esta es sobrescrita con el valor de la segunda, y luego permite restituir ese valor guardado en la segunda celda. Sin esta variable auxiliar, uno de los dos valores se perdería de forma irrecuperable durante el proceso de intercambio.

Pregunta 5. Autoevaluación: reto más complejo y forma de resolución

A continuación, cada integrante del grupo expone de manera individual cuál de los retos abordados le resultó más complejo de comprender a bajo nivel y de qué manera logró resolverlo.

Sofia Mojica

El reto que me resultó más complejo fue entender el desbordamiento de pila en la recursividad, porque no lograba visualizar cómo se apilaban las llamadas en la memoria. Lo resolví dibujando en papel cada marco de pila con sus variables locales a medida que la función se llamaba a sí misma, lo que me permitió ver claramente por qué el programa se detenía de forma abrupta al no existir un caso base.

Gabriel Zacarías

Para mí, el punto más difícil fue comprender por qué `break` realmente mejora el rendimiento y no solo detiene el bucle. Lo resolví ejecutando el mismo algoritmo de búsqueda con y sin `break`, contando manualmente el número de iteraciones que realizaba el procesador en cada caso, lo cual me permitió comprobar la diferencia de eficiencia de forma práctica.

Mateo Roberts

El desafío que más trabajo me llevó fue el direccionamiento de arreglos, ya que al principio no entendía por qué las direcciones de memoria aumentaban de a un número fijo. Logré resolverlo imprimiendo las direcciones de cada elemento del arreglo en el compilador y comparándolas manualmente con el tamaño del tipo de dato utilizado.

Leonel De Armas

Lo que más me costó comprender fue el intercambio de valores mediante punteros, porque confundía el puntero con el valor al que apuntaba. Lo pude resolver haciendo un seguimiento paso a paso del algoritmo, escribiendo en cada línea el contenido de la variable temporal y de las celdas de memoria involucradas hasta entender el orden correcto de las asignaciones.

Conclusión

El desarrollo de este cuestionario permitió al grupo comprender que, por debajo de la sintaxis de un lenguaje de programación, existen mecanismos concretos de administración de memoria RAM que determinan el correcto o incorrecto funcionamiento de un programa. Comprender el caso base en la recursividad, el impacto de la instrucción `break` en el rendimiento, la contigüidad de los arreglos en memoria y el rol de las variables temporales en el manejo de punteros resulta fundamental para escribir código eficiente, seguro y libre de errores críticos como el desbordamiento de pila o la pérdida de datos durante un intercambio de valores.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.).

Deitel, P., & Deitel, H. (2016). *Cómo programar en C* (8.ª ed.). Pearson Educación.

Joyanes Aguilar, L. (2018). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos* (5.ª ed.). McGraw-Hill.

Kernighan, B. W., & Ritchie, D. M. (1991). *El lenguaje de programación C* (2.ª ed.). Prentice Hall.

Apéndice

Repositorio del proyecto

El código del grupo se encuentra disponible en el siguiente repositorio:

https://github.com/matuteXEN6/repasogeneral.git